

Exercise 03

```
vihan@vihanzVM in ~ via C v16.1.1-gcc via v5.42.2 via v3.14.5 took 18s702ms
$ gcc -fopenmp -O2 -o exe3_master exe3_master.c && OMP_NUM_THREADS=1 ./exe3_master
thread # 0
Inside Master # 0
Exiting Master
Hi again from thread # 0
```

```
vihan@vihanzVM in ~ via C v16.1.1-gcc via v5.42.2 via v3.14.5
$ gcc -fopenmp -O2 -o exe3_master exe3_master.c && OMP_NUM_THREADS=2 ./exe3_master
thread # 0
Inside Master # 0
Exiting Master
Hi again from thread # 0
thread # 1
Hi again from thread # 1
```

```
vihan@vihanzVM in ~ via C v16.1.1-gcc via v5.42.2 via v3.14.5
$ gcc -fopenmp -O2 -o exe3_master exe3_master.c && OMP_NUM_THREADS=4 ./exe3_master
thread # 2
Hi again from thread # 2
thread # 3
Hi again from thread # 3
thread # 0
Inside Master # 0
Exiting Master
Hi again from thread # 0
thread # 1
Hi again from thread # 1
```

```

vihan@vihanVM in ~ via C v16.1.1-gcc via v5.42.2 via v3.14.5
$ gcc -fopenmp -O2 -o exe3_master exe3_master.c && OMP_NUM_THREADS=8 ./exe3_master
thread # 2
Hi again from thread # 2
thread # 4
Hi again from thread # 4
thread # 5
Hi again from thread # 5
thread # 6
Hi again from thread # 6
thread # 3
Hi again from thread # 3
thread # 0
Inside Master # 0
Exiting Master
Hi again from thread # 0
thread # 7
thread # 1
Hi again from thread # 1
Hi again from thread # 7

```

Q. Briefly describe how the pragma omp master command works in this program.

#pragma omp master restricts a block to run only on the master thread (thread 0) never any other thread, regardless of timing. My output confirms this: "Inside Master" only appears attached to thread # 0. The key difference from single: master has no implicit barrier, so other threads don't wait for it they can print their "Hi again" line before or after the master block runs. Compare that to my actual output above threads 1, 2, 3 all finished before thread 0 even started its master block.